Project Report

Enterprise Financial Data Platform (EFDP)

MACSE503 Data Engineering

Team Members

Avirup Das (26MML1005)

T Alagupandi (26MML1013)

Y Aditya Venkat (26MML1025)



School of Computer Science and Engineering (SCOPE)

Vellore Institute of Technology - Chennai

Abstract

The Enterprise Financial Data Platform (EFDP, UC20) builds a unified, curated foundation for financial reporting from ten SAP finance and master-data extracts holding 1,465,129 rows. Over four sprints the team defined the business scope and a Bronze/Silver/Gold architecture (Sprint 0), documented the data and loaded it into PostgreSQL staging with ten Pentaho pipelines that read CSV, Excel, XML and JSON files (Sprint 1), profiled and quality-scored every table and built a star-schema warehouse of one fact table and nine dimensions (Sprint 2), and published an EFDP Power BI dashboard on top of it (Sprint 3). The dashboard reports 332K line items and 2.46T in both total debit and total credit. The quality assessment found that no table passes every rule: duplicate rows are concentrated in SKAT, LFA1 and T001, and SKAT.chart_of_accounts is empty in all 782,051 rows. These issues, and the need to reconcile the dashboard with SQL, define the next steps.

Keywords: data warehouse, star schema, Pentaho Data Integration, PostgreSQL, SAP finance data, data quality, Power BI

1. Introduction

Large enterprises keep financial data in several systems and file formats. Document headers and line items, G/L accounts, cost centers, customers, vendors and company codes are held in separate tables, extracted as CSV, Excel, XML or JSON. When these are analysed one by one, reports depend on duplicate rows, empty fields and column types or names that differ from table to table, and joins can silently mismatch.

EFDP addresses this by moving the data through a repeatable, layered pipeline. Raw SAP extracts are landed unchanged in PostgreSQL staging, described in a data dictionary, profiled and scored against quality rules, and then reorganised into a star schema that Power BI reads. Every step is built with Pentaho Data Integration, Python/pandas and SQL and kept under Git version control.

The project was delivered in four Agile sprints, and this report follows that order. Section 2 gives the objectives, scope and technology stack. Sections 3 to 6 describe Sprints 0 to 3, each with its activities, results and screenshots. Section 7 maps deliverables to sprints, Section 8 lists limitations and next steps, and Section 9 concludes.

2. Project overview

EFDP integrates financial and master data from several independent enterprise systems into one curated foundation for financial reporting and analysis.

Objectives

- Unify core data: bring finance and master data into a single PostgreSQL platform.
- Ensure governance: keep metadata, a data dictionary and traceable source-to-warehouse mappings.
- Power analytics: standardise the data for financial reporting and executive dashboards.
- Build collaborative foundations: Git/GitHub version control and enterprise-level documentation.

Scope. Source discovery, raw loading into PostgreSQL staging, profiling and quality assessment, star-schema design and load, and Power BI reporting. Real-time ingestion, scheduled incremental loads and online deployment are not part of Sprints 0 to 3.

Technology stack

Component	Technology	Purpose
ETL	Pentaho Data Integration	Data ingestion and ETL transformations
Database	PostgreSQL	Staging and enterprise data storage
Programming	Python / Pandas	Data processing and profiling
Version control	Git / GitHub	Collaboration and version control
Reporting	Power BI	Business dashboards and reporting
Documentation	Markdown / MS Word	Technical documentation

Component	Technology	Purpose
Methodology	Agile Scrum	Sprint-based project development

3. Sprint 0: Project initiation and architecture

Sprint 0 fixed the business understanding, scope and layered design before any pipeline was built.

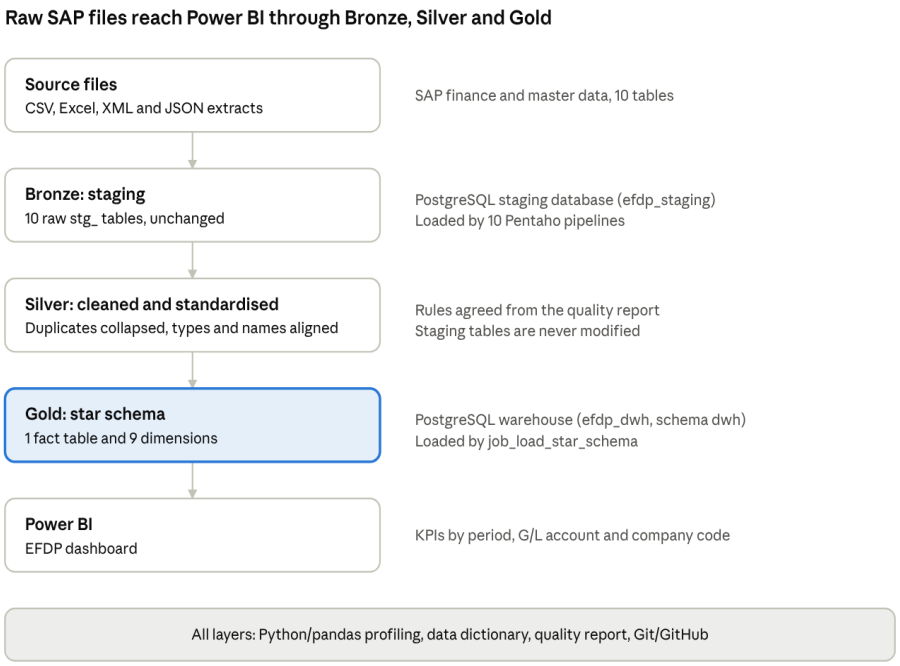
Objective. Understand the enterprise finance ecosystem and design the Enterprise Financial Data Platform before development begins.

Key activities

- Study enterprise finance processes and the broader ERP landscape.
- Identify business stakeholders and financial reporting requirements.
- Prepare the Business Requirements Document (BRD).
- Identify financial and master data sources across the organisation.
- Define project scope, objectives and success criteria.
- Design the high-level platform architecture.

Project setup

- Create the Git repository and an enterprise repository structure.
- Prepare the sprint backlog.
- Design the solution architecture as a layered Bronze/Silver/Gold model.
- Prepare the project charter.
-



Deliverables: Business Requirements Document, Solution Architecture, Git repository, Sprint Backlog and Project Charter.

Solution architecture. Raw SAP extracts land unchanged in Bronze (PostgreSQL staging). Rules agreed in the quality report are then applied to separate cleaned data, never to the staging tables. Gold holds the star schema that Power BI reads.

EFDP layered architecture: Bronze, Silver and Gold

Each layer is stored and loaded separately, so a problem can be traced to the layer where it started.

4. Sprint 1: Data discovery and ingestion

Sprint 1 built the financial data landing zone: ten SAP-derived tables were documented, then loaded as raw data into PostgreSQL staging through ten Pentaho pipelines.

Objective. Integrate heterogeneous financial and master data from multiple enterprise systems and load it, unchanged, into PostgreSQL staging.

Key activities

- Prepare the data dictionary with field descriptions, data types and examples.
- Validate business keys and relationships across source systems.
- Build Pentaho transformations for the supplied source files.
- Load raw data into the PostgreSQL staging tables.
- Commit successful ingestion pipelines to GitHub.

4.1 Source data and data dictionary

The data dictionary documents every column of the ten staging tables, with its PostgreSQL type, maximum length, business description, record grain and relationships.

Staging table	SAP table	Description	Module	Columns
stg_bkpf	BKPF	Accounting document header	FI	15
stg_bseg	BSEG	Accounting document line items	FI	21
stg_csks	CSKS	Cost center master data	CO	9
stg_cskt	CSKT	Cost center texts	CO	7
stg_kna1	KNA1	Customer master, general data	FI-AR	12
stg_lfa1	LFA1	Vendor master, general data	FI-AP	12
stg_ska1	SKA1	G/L account master	FI-GL	11
stg_skat	SKAT	G/L account texts	FI-GL	10
stg_t001	T001	Company code master	Enterprise structure	9

Staging table	SAP table	Description	Module	Columns
stg_t002	T002	SAP system languages	Basis / reference	7

Key relationships recorded in the dictionary:

- stg_bkpf joins stg_bseg on company_code, accounting_document_no and fiscal_year.
- stg_bseg links to G/L accounts (stg_ska1, stg_skat), customers (stg_kna1), vendors (stg_lfa1) and cost centers (stg_csks, stg_cskt).
- company_code values map to stg_t001; language_code values map to stg_t002.
- Keys are documented as business identifiers. No primary or foreign key constraints exist in staging.

4.2 ETL pipelines

Each pipeline follows the same pattern: Extract from the source file, Transform with Pentaho steps to prepare the records, Load into a staging table. Four source formats were supported: CSV, Excel, JSON and XML.

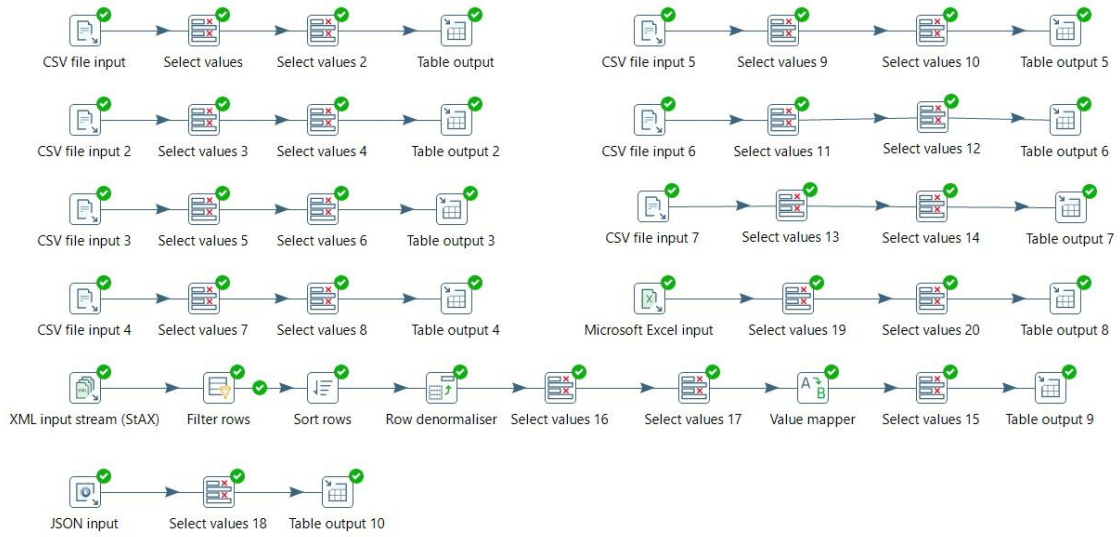


Figure 1. Pentaho ingestion transformations. Every step shows a green tick, meaning it ran successfully.

The ten pipelines in Figure 1 are:

- Seven CSV pipelines: CSV file input, then two Select values steps, then Table output (CSV file input 1 to 7).
- One Excel pipeline: Microsoft Excel input, two Select values steps, Table output 8.
- One XML pipeline: XML input stream (StAX), Filter rows, Sort rows, Row denormaliser, Select values, Value mapper, Select values, Table output 9. This is the longest chain; the Row denormaliser step turns repeated rows into columns.
- One JSON pipeline: JSON input, Select values, Table output 10.

Deliverables: Source inventory, data dictionary (EFDP_Data_Dictionary.md), Pentaho ETL pipelines, PostgreSQL staging database with ten stg_ tables, and the Git commit history of the ingestion code.

5. Sprint 2: Data profiling, data quality and data warehouse

Sprint 2 turned the raw staging layer into trusted, analytics-ready data: the ten tables were profiled and quality-scored, then a star schema was designed and populated.

Objective. Profile and quality-check the ten SAP staging tables, then design and build a star-schema data warehouse.

Process

1. Profile: read-only checks on every table (column types, row counts, NULLs and blanks, exact duplicate rows, numeric statistics).
2. Assess quality: score each table on completeness, uniqueness, validity and consistency; rate each failed rule Low (under 1% of rows affected), Medium (1 to 5%) or High (over 5%).
3. Clean and standardise: apply agreed rules to a separate cleaned dataset; the staging tables are not changed.
4. Model: one fact table (fact_accounting_line_item) and its dimensions.
5. Load: load the dimensions first, then the fact table, and reconcile row counts.

5.1 Data profiling

All ten tables were profiled with the same Jupyter notebook (pandas and SQLAlchemy, SELECT queries only), so nothing in staging was changed. The ten tables hold 1,465,129 rows.

Table	Description	Rows	Cols	Cols with NULLs	Dup. rows	Dup. %	Rows in groups
BKPF	Document header	150,057	15	0	470	0.31	921
BSEG	Document line items	332,106	21	13	77	0.02	154
CSKS	Cost center master	3,353	9	5	40	1.19	79
CSKT	Cost center texts	26,977	7	2	2	0.01	4
KNA1	Customer master	20,209	12	8	23	0.11	42
LFA1	Vendor master	3,035	12	9	389	12.82	488
SKA1	G/L account master	146,692	11	4	0	0.00	0
SKAT	G/L account texts	782,051	10	4	71,924	9.20	126,461
T001	Company codes	605	9	5	67	11.07	92
T002	Language keys	44	7	0	0	0.00	0
Total		**1,465,129**			**72,992**		

Figure 2. Profiling output: rows, columns, columns with NULLs and exact duplicates per table.

What the profiling showed:

- No table has a blank string; missing information is stored as NULL.

- 72,992 rows repeat an earlier row, and 98.5% of them (71,924) are in SKAT. LFA1 (12.82%) and T001 (11.07%) have the next-highest duplicate rates; SKA1 and T002 have none.
- Two columns are empty in every row: BSEG.tax_code and SKAT.chart_of_accounts. BSEG.special_gl_indicator has a value in only 5 rows.
- BSEG amounts are very skewed: minimum 0.03, maximum about 98.98 billion, and a mean roughly 250 times the median. No amount is negative, so the sign probably comes from debit_credit_indicator.
- Types and names differ between tables: client_id is text in seven tables but BIGINT in KNA1 and LFA1; dates are stored as DATE, TIMESTAMP or text; gl_account_no is gl_account_number in SKAT; BSEG has the misspelt column payement_baseline_date.

The date-profiling and candidate-key sections of the notebook were left empty when it was run, so date ranges and business-key uniqueness are not covered by this report.

5.2 Data quality assessment

Because no business rules were supplied, the rules were defined from column names and SAP conventions and should be confirmed with the data owners. Completeness is the average share of populated mandatory fields; uniqueness is 100 minus the share of exact duplicate rows.

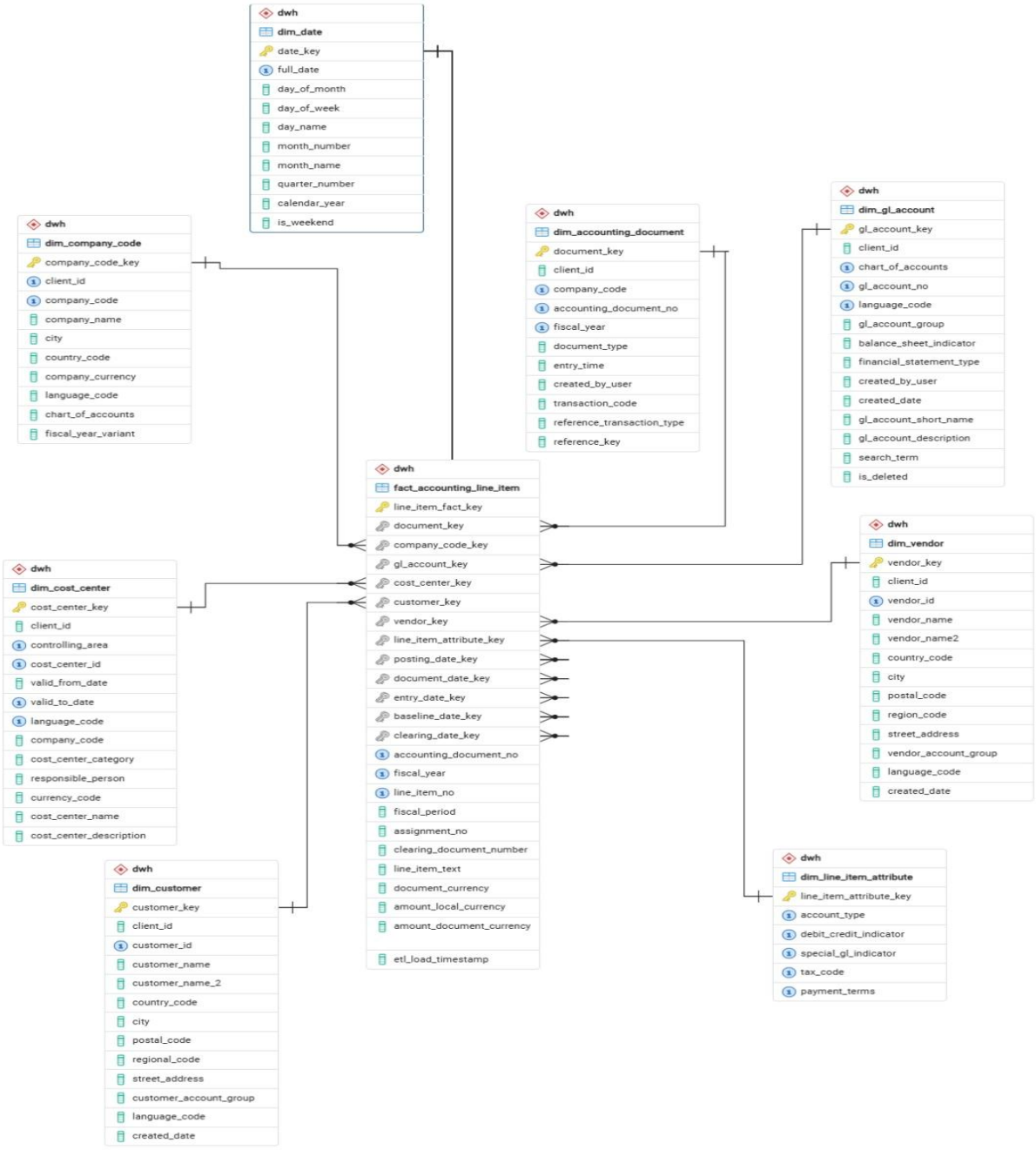
Table	Rows	Mandatory fields failed	Completeness %	Uniqueness %	Highest severity
BKPF	150,057	0 of 15	100.00	99.69	Low
BSEG	332,106	2 of 5	99.99	99.98	Low
CSKS	3,353	3 of 3	99.97	98.81	Medium
CSKT	26,977	1 of 1	98.95	99.99	Medium
KNA1	20,209	3 of 3	99.74	99.89	Low
LFA1	3,035	3 of 3	97.17	87.18	High
SKA1	146,692	2 of 2	97.23	100.00	Medium
SKAT	782,051	3 of 3	65.09	90.80	High
T001	605	3 of 3	97.36	88.93	High
T002	44	0 of 7	100.00	100.00	Low

Figure 3. Quality scores by table: mandatory fields failed, completeness, uniqueness and highest severity.

- Of 45 mandatory fields checked, 20 contain NULLs. Only BKPF and T002 have fully populated mandatory fields; only SKA1 and T002 are free of duplicates. No table passes every rule.
- Weakest table: SKAT, with 65.09% completeness (chart_of_accounts is empty in all 782,051 rows) and 90.80% uniqueness.
- LFA1 (87.18%) and T001 (88.93%) have uniqueness below 90%. LFA1.city is missing for 8.24% of vendors.
- Validity: fiscal_period stays within 1 to 12 and amounts are above zero (both pass). The very large BSEG amounts need review because no upper limit was defined. Seven text columns that should be dates or timestamps, and T002.is_deleted stored as text, fail.
- Consistency: client_id type, date storage, is_deleted type and field names all fail across tables, which will cause silent mismatches in joins.

Priority issues, in order: (1) SKAT duplicates and empty chart_of_accounts; (2) LFA1 duplicates and missing city; (3) T001 duplicates and NULLs in a reference table other tables depend on; (4) SKA1 audit fields, CSKT names and CSKS duplicates; (5) extreme BSEG amounts; (6) mismatched types and names that affect every join.

5.3 Star schema design



The warehouse (schema dwh in PostgreSQL) has one fact table, fact_accounting_line_item, and nine dimensions. Its grain is one accounting line item: company code, document number, fiscal year and line item number, enforced by a unique constraint. Every dimension has a surrogate key, and the fact table carries foreign keys to seven of them plus five role-playing date keys (posting, document, entry, baseline and clearing date).

Figure 4. Star schema ERD: fact_accounting_line_item at the centre with its dimensions.

Warehouse table	Built from (staging)	What it holds
fact_accounting_line_item	stg_bseg joined to stg_bkpf	Amounts in local and document currency, fiscal year and period, clearing document, line text, load timestamp
dim_accounting_document	stg_bkpf	Document type, entry time, creator, transaction code, reference
dim_company_code	stg_t001	Company name, city, country, currency, chart of accounts, fiscal year variant
dim_gl_account	stg_ska1 + stg_skat	Account group, balance sheet indicator, statement type, names and description
dim_cost_center	stg_csks + stg_cskt	Controlling area, validity dates, category, responsible person, names
dim_customer	stg_kna1	Customer name, address, account group
dim_vendor	stg_lfa1	Vendor name, address, account group
dim_line_item_attribute	stg_bseg	Distinct combinations of account type, debit/credit indicator, special G/L indicator, tax code, payment terms
dim_language	stg_t002	SAP and ISO language codes (reference table, no foreign key from the fact)
dim_date	generated	Calendar from the earliest to the latest date in the data, plus one

Warehouse table	Built from (staging)	What it holds
		year either side

Design choices that answer the profiling findings:

- Each dimension is built from one row per business key, so the exact duplicates found in SKAT, LFA1, T001 and CSKS are not copied into the warehouse.
- Text dates in BSEG (clearing_date, payement_baseline_date) are converted to date keys only when they match a valid date pattern.
- The debit/credit indicator is kept in dim_line_item_attribute because the amounts themselves are never negative.
- Staging stays untouched; all cleaning happens in the transformations that populate dwh.

5.4 Warehouse load with Pentaho

The warehouse is populated by ten Pentaho transformations (nine dimensions and one fact) run in order by one job, job_load_star_schema. Source and target connections come from kettle.properties: the staging database (efdp_staging, tables public.stg_*) and the warehouse database (efdp_dwh, schema dwh).

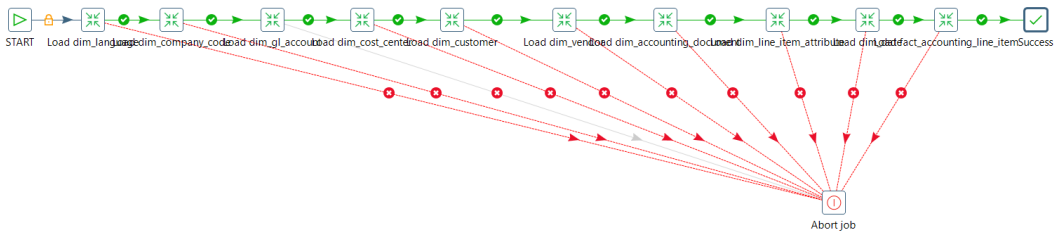


Figure 5. job_load_star_schema: dimensions load first, then the fact table, ending in Success. The red hops lead to Abort job.

- **Load order:** dim_language, dim_company_code, dim_gl_account, dim_cost_center, dim_customer, dim_vendor, dim_accounting_document, dim_line_item_attribute, dim_date, then fact_accounting_line_item, so every surrogate key exists before the fact table looks it up.
- **Dimensions:** each transformation reads staging with a SELECT that keeps one row per business key (the latest record) and writes it with an Insert/Update step, so the job can be re-run without duplicating rows.
- **Fact table:** BSEG lines are de-duplicated on company code, document number, fiscal year and line item; joined to the BKPF header for posting date, fiscal period and currency; and matched to the dimensions through lookup steps. Rows with no header or no company code go to a rejected-rows step instead of the fact table.
- **Failure handling:** every transformation has a failure hop to Abort job, which stops the run with the message "Star schema load failed. See the log of the last transformation."

Sprint 2 deliverables: data profiling report, data quality report, star schema (fact and dimension model with the staging-to-warehouse mapping, starSchema.sql), Pentaho warehouse transformations and job, and the populated PostgreSQL data warehouse.

6. Sprint 3: Visual analytics and governance

Sprint 3 put the warehouse to use: the EFDP Power BI dashboard reads the star schema and reports line-item volume, debit and credit totals by period, G/L account and company code.

6.1 EFDP dashboard

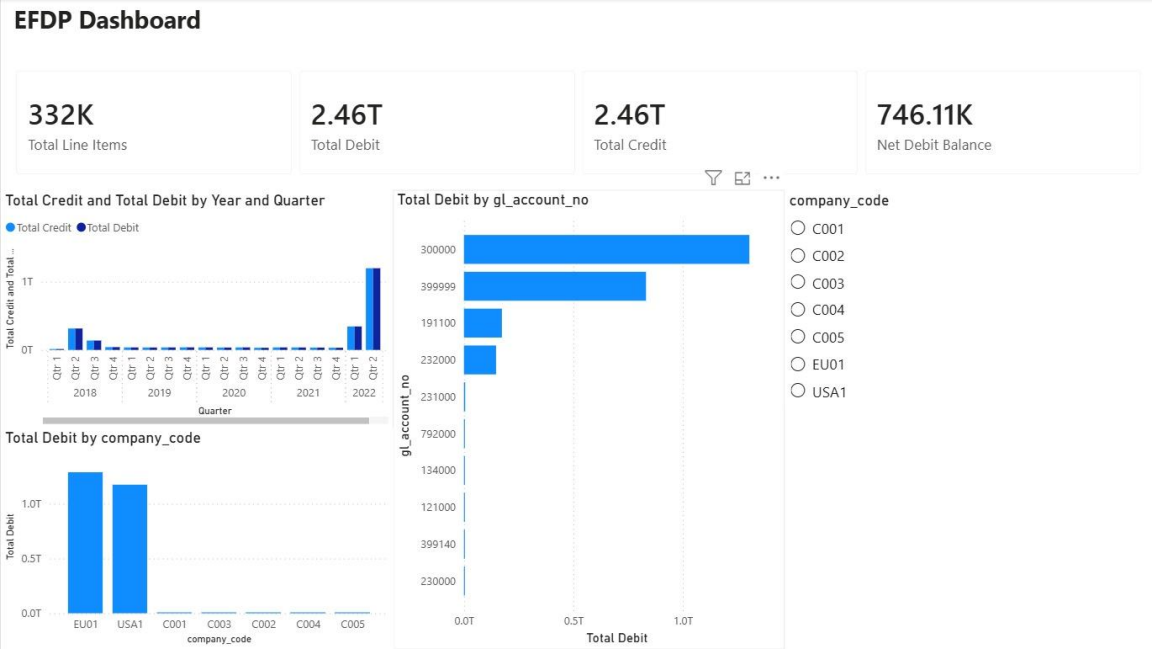


Figure 6. EFDP dashboard: KPI cards, debit and credit by year and quarter, debit by G/L account and by company code, and a company_code slicer.

Visual	What it shows (values read from Figure 6, chart values approximate)
KPI cards	332K total line items; 2.46T total debit; 2.46T total credit; 746.11K net debit balance
Total credit and total debit by year and quarter	Both series move together. Quarters from 2018 to 2021 are small; 2022 Q1 is about 0.3T and 2022 Q2 is the peak at about 1.2T
Total debit by gl_account_no	Account 300000 is largest (about 1.3T), then 399999 (about 0.8T), 191100 and 232000 (about 0.15T each); the other accounts are close to zero
Total debit by company_code	EU01 (about 1.3T) and USA1 (about 1.2T) carry almost all of the debit; C001 to C005 are very small

Visual	What it shows (values read from Figure 6, chart values approximate)
company_code slicer	Filters every visual to one company code (C001 to C005, EU01, USA1)

6.2 Reading the results

- **The books balance.** Total debit and total credit are both 2.46T, and the net debit balance of 746.11K is about 0.00003% of that, which is what double-entry postings should produce.
- **Volume matches staging.** 332K line items is consistent with the 332,106 rows of stg_bseg in the profiling report.
- **Totals are driven by a few very large postings.** The profiling report found BSEG amounts up to about 98.98 billion with a mean about 250 times the median, and the dashboard shows two company codes and a handful of G/L accounts holding nearly all value. These figures should be confirmed with the finance team before being used for decisions.

6.3 Governance and traceability

The following controls exist in the project artifacts:

- Data dictionary for all ten staging tables, with grain, identifiers and relationships.
- Documented staging-to-warehouse mapping (section 5.3 and the header of starSchema.sql).
- Data quality report with severity ratings, priority issues and recommendations.
- Repeatable loads: upsert-based dimension loads, a rejected-rows stream for fact lines without a header or company code, a job that aborts on any failed step, and an etl_load_timestamp column on every fact row.
- Git/GitHub for version control of ETL, SQL and documentation.

Sprint 3 deliverables: EFDP Power BI dashboard, the star-schema load job and transformations (Pentaho), and the governance documentation listed above.

7. Deliverables traceability

Sprint	Deliverable	Purpose / evidence
0	Business Requirements Document	Business problem, stakeholders, reporting requirements
0	Solution architecture (Bronze/Silver/Gold)	End-to-end data flow
0	Git repository, sprint backlog, project charter	Collaboration, planning, scope
1	Source inventory	Financial and master data sources found in discovery

Sprint	Deliverable	Purpose / evidence
1	EFDP_Data_Dictionary.md	Columns, types, grain and relationships of 10 staging tables (section 4.1)
1	Pentaho ingestion transformations	10 pipelines loading CSV, Excel, XML and JSON (Figure 1)
1	PostgreSQL staging database	Ten stg_ tables holding raw data
2	Data profiling report	Structure, completeness and duplication (Figure 2)
2	Data quality report	Rule-based scores, severity, priorities (Figure 3)
2	starSchema.sql	DDL for 1 fact and 9 dimensions in schema dwh (Figure 4)
2	efdp_pentaho_star_schema (10 .ktr files, job_load_star_schema.kjb, kettle.properties.sample)	Populates the warehouse (Figure 5)
3	EFDP Power BI dashboard	KPIs and analysis by period, G/L account and company (Figure 6)
3	Governance documentation	Dictionary, mapping, quality findings, load controls
All	Git/GitHub history	Version control and reproducibility

8. Limitations and next steps

The findings below come from the profiling and quality reports and from the build artifacts; none of the suggested fixes has been applied to the staging tables.

Known limitations

- The quality rules were defined for the report, not supplied by the business, and need confirming with data owners.
- Date profiling and the business-key uniqueness check were not run, so text dates and key duplicates are unverified.
- Only exact duplicate rows were counted; rows that share a key but differ elsewhere were not examined.
- Staging row counts were not compared with the source systems, and relationships between tables (for example every BSEG document existing in BKPF) were not tested in the profiling stage.
- BSEG tax_code and SKAT chart_of_accounts are empty; it is not known whether they are unused or were lost in extraction.
- Dashboard figures have not been reconciled against SQL queries, and the very large BSEG amounts are unexplained.
- EFDP is a project pipeline: scheduled incremental loads and online deployment are out of scope.

9. Conclusion

Across Sprints 0 to 3, EFDP delivered a traceable path from raw SAP extracts to a financial dashboard. Sprint 0 fixed the scope and the Bronze/Silver/Gold design; Sprint 1 documented ten tables and loaded them through ten Pentaho pipelines; Sprint 2 profiled and quality-scored the data and populated a star schema of one fact table and nine dimensions through a single orchestration job; and Sprint 3 turned the warehouse into the EFDP dashboard.

The results are useful but not yet final. The dashboard balances (debit and credit both 2.46T) and its 332K line items are consistent with the staging data, yet no staging table passes every quality rule, the largest BSEG amounts are unexplained, and the quality rules still need approval from the data owners. EFDP is therefore a sound foundation for financial reporting, and the next step is to resolve the high-severity findings, confirm business keys, and reconcile staging, warehouse and Power BI before the figures are used for decisions.